

Project Design

Trivia game

Members:

- Owen Winegar
 - Alex Ferguson
 - Ryan Howard
-

1. Project idea

Our group will create a multiplayer trivia quiz application that works similarly to a Kahoot-style game. The system will use the client-server architecture we learned in lecture, where multiple clients connect to a central server through a network.

The server will host the trivia game, store questions, and manage player scores. Players will connect using the client program, join the game, and answer trivia questions. The server will send questions to all connected players, collect their answers, evaluate correctness, and update the scoreboard appropriately. If players have the same answer, the person who answered earliest will receive the most points.

At the end of the game, the server will display the final rankings of all players.

2. Requirement analysis

Application Usage

1. The server program is started by the host.
2. Players start the client program, which connects to the server.
3. Each player enters a username.
4. Once players have joined, the host begins the trivia game on the server.

5. The server sends questions to all connected clients and clients are given 10 seconds to respond.
6. Clients submit their answers, with their username, so points can be distributed correctly.
7. The server evaluates the answers and updates the scoreboard.
 - a. If more than one player has the correct answer, points will be reduced as the time to answer the question increases.
8. The updated scores are sent to all players.
9. After all questions are answered, the server sends the final rankings.

- Allow the user to exit the application

Client Functions

The client program will perform the following functions:

- Connect to the trivia server using an IP address and port number
- Allow the player to enter a username
- Receive trivia questions from the server
- Display questions and answer choices
- Send the selected answer to the server
- Receive score updates from the server
- Display the final leaderboard at the end of the game, sent from the server

Server Functions

The server program will perform the following functions:

- Accept connections from multiple clients
- Store connected player usernames
- Maintain the list of trivia questions
- Send questions to all connected clients
- Receive answers from each client
- Check whether answers are correct
- Update player scores
- Send updated scoreboards to all clients

- Send final rankings when the game ends

Client Interface example:

Question 1:

What is the capital of France?

- A) London
- B) Berlin
- C) Paris
- D) Madrid

Enter answer: C

The server will keep track of the scores, like:

Scoreboard

Ryan: 3

Alex: 2

Jamie: 1

3. Protocol Design

The communication between client and server will use a text-based protocol. Messages will contain a command followed by parameters.

Client → Server Messages

Join Game

JOIN <username>

Example:

JOIN Ryan

Purpose:

Registers a player with the server.

Submit Answer

ANSWER <question_number> <choice>

Example:

ANSWER 1 C

Purpose:

Sends the player's selected answer to the server.

Exit Game

EXIT

Purpose:

Disconnects the client from the server.

Server → Client Messages

Welcome Message

WELCOME <username>

Example:

WELCOME Ryan

Purpose:

Confirms that the player successfully joined the game.

Question Message

QUESTION <number>

<question text>

- A) option
- B) option
- C) option
- D) option

Example:

QUESTION 1

What is the capital of France?

- A) London
- B) Berlin
- C) Paris
- D) Madrid

Purpose:

Sends a trivia question to all connected clients.

Scoreboard Update

SCORES

Alex 3

Owen 2

Ryan 1

END

Purpose:

Updates all players with the current scores.

Game Over Message

GAME_OVER

1 Alex 5

2 Owen 4

3 Ryan 2

Purpose:

Displays final rankings to all players.

Client Processing (you can think of these like methods)

When the client receives messages from the server:

WELCOME

- Display confirmation message.

QUESTION

- Display the question and answer options.
- Prompt the user to select an answer.
- Send an ANSWER message to the server with associated time.

SCORES

- Display updated scoreboard.

GAME_OVER

- Display final rankings.
 - End the game.
-

Server Processing

When the server receives messages from clients:

JOIN

1. Register the player's username.
2. Add the player to the player list.
3. Send a WELCOME message.

ANSWER

1. Record the player's answer.
2. Compare it with the correct answer.
3. Update player score if correct.
 - a. If more than one player has the correct answer, points will be reduced as the time to answer the question increases.

EXIT

1. Remove the player from the game.
2. Close connection.

The server will send questions sequentially and wait until all players have answered before continuing to the next question.

4. Work Division Among Group Members

Owen

- Implement the client application
- Handle displaying questions and sending answers
- Implement client-server communication

Ryan

- Implement server networking
- Handle client connections and communication

Alex

- Implement game logic on the server side

- Manage trivia questions and the scoring system
- Generate scoreboard and final results

All group members will assist with testing and debugging the application.

5. Stretch Goal

You might want to expand this administration functionality, like configuring number of questions, number of participants etc. Make it closer to Kahoot.

Our stretch goal will be to expand the administration functionality, such as configuring the number of questions, the number of questions to ask, and the maximum number of participants. Since this is more complicated, our group will only complete this task if we have enough time in our schedule to allow it.